

NERV V2.0 - Expanded Appendix B: End-to-End Transaction Lifecycle & Mathematical Formalism

Phase 1: Client-Side Transaction Construction

Business/Functional Process

The transaction lifecycle begins when a user, Alice, decides to transfer NERV tokens to another user, Bob. This process is entirely client-side and occurs in the user's wallet application, which can be a mobile app, desktop application, or hardware wallet. The wallet maintains a local database of unspent private notes (UTXO-style shielded outputs) that represent Alice's available balance.

Alice initiates the transaction by:

1. Entering Bob's wallet address (which is a blinded public key derived from his master secret)
2. Specifying the amount (v nano-NERV) she wishes to send
3. Approving the transaction via biometric authentication, PIN, or hardware wallet confirmation

The wallet then performs a series of cryptographic operations entirely offline or in a trusted execution environment on the device:

Step 1: Private Note Selection

The wallet scans its local database of unspent private notes. Each note contains a commitment to a value and a blinding factor, along with the recipient's public key. The wallet must select a set of notes $\{n_1, n_2, \dots, n_\ell\}$ such that their total value is at least the amount being sent (v) plus the transaction fee (fee). This is analogous to selecting UTXOs in Bitcoin, but done privately.

If the total value of selected notes exceeds $(v + fee)$, the wallet creates a "change note" for Alice herself, ensuring the total input equals total output.

Step 2: Output Construction

The wallet constructs two new private notes:

- Bob's note: Encrypted with Bob's public key, containing the amount v
- Alice's change note: Encrypted with Alice's own public key, containing the remainder ($\text{sum}(\text{selected notes}) - v - \text{fee}$)

Each note also includes a nullifier (a unique identifier derived from the note's secret key and a random nonce) that will be used to prevent double-spending. The nullifier is computed as a hash of the note's secret key and a unique identifier, ensuring that once a note is spent, its nullifier can be published on-chain to mark it as spent without revealing the note's contents.

Step 3: State Delta Calculation

The wallet computes the scalar state delta ΔS , which represents the change in the ledger's key-value state:

$$\Delta S = v_{\text{bob}} + v_{\text{change}} - \sum_{i \in I} v_i$$

Where:

- v_{bob} is the amount sent to Bob
- v_{change} is the change amount returned to Alice
- $\sum_{i \in I} v_i$ is the total value of the input notes selected

In a correctly constructed transaction, $\Delta S = 0$, meaning value is conserved (inputs equal outputs). However, the transaction fee may make this slightly negative (fee is paid to validators).

Step 4: Nullifier Publication

The wallet computes the nullifier for each input note. These nullifiers are included in the transaction to prevent double-spending. The nullifier for a note is computed as:

$$\text{nullifier} = \text{BLAKE3}(\text{secret_key} \parallel \text{note_id} \parallel \text{nonce})$$

The secret key is known only to the owner of the note, so only the rightful owner can spend it. Once a nullifier is published, the network records it as spent, preventing any future transaction from using the same note.

Mathematical Formalism

Commitment Scheme (Pedersen-style)

Each private note is represented as a commitment:

$$C = v \cdot G + r \cdot H$$

Where:

- v is the value (amount) of the note
- r is a random blinding factor (chosen uniformly from the field)
- G and H are fixed generator points on the BLS12-381 curve (or a post-quantum alternative)
- C is the resulting commitment point

This is a Pedersen commitment, which is perfectly hiding (the commitment reveals nothing about v) and computationally binding (it's infeasible to find different (v, r) that produce the same C).

Note Structure

Each private note contains:

1. The commitment C
2. The nullifier $\text{nullifier} = \text{BLAKE3}(\text{secret_key} \parallel \text{nonce})$
3. The recipient's public key (blinded)
4. Encrypted metadata (amount, memo, etc.)

The note is encrypted using the recipient's public key via ML-KEM-768, ensuring only the intended recipient can decrypt and spend it.

Homomorphic Delta Computation

Given that NERV V2.0 uses a linear Perceptron for state encoding, the transaction delta δ_{tx} is computed exactly as:

$$\delta_{\text{tx}} = W \cdot \Delta S + b_{\text{tx}}$$

Where:

- $W \in \mathbb{R}^{(64 \times N)}$ is the network's public weight matrix (N features mapped to 64 dimensions)
- $\Delta S \in \mathbb{R}^N$ is the state delta vector (the scalar ΔS expanded to N features)
- $b_{\text{tx}} \in \mathbb{R}^{64}$ is a transaction-specific bias derived from the nullifiers (prevents replay attacks)

The dimension N represents the number of features in the state representation. In practice, N may be ~ 256 features (e.g., account identifiers, timestamp, transaction type, fee, etc.). The 64-dimensional output is the embedding delta that will be added to the canonical state embedding.

Zero-Knowledge Witness Assignment

The wallet prepares the following private witnesses for the Halo2 circuit:

Witness	Description
W	Public weight matrix (public input, not a witness)
ΔS	Private state delta vector
b_{tx}	Private transaction bias
δ_{tx}	Computed output delta (private, but will be revealed as part of proof)

The circuit proves that: $\delta_{tx} = W \cdot \Delta S + b_{tx}$ without revealing ΔS or b_{tx} .

Circuit Constraints (Approximate)

For a 64-dimensional output with $N=256$ input features:

1. Dot Product Gates: $N \times 64 = 16,384$ multiplication and addition gates
2. Bias Addition Gates: 64 addition gates
3. Homomorphic Update Gate: 64 addition gates (verifying $e_{t+1} = e_t + \delta_{tx}$)
4. Nullifier Verification Gate: ~ 100 gates for checking nullifier uniqueness

Total: $\sim 17,000$ constraints, well within the $\sim 50,000$ constraint budget.

Threshold Encryption of Transaction Payload

The complete transaction payload (commitments, nullifiers, encrypted notes, and the ZK

proof π) is encrypted using the network's Distributed Key Generation (DKG) collective public key PK_{DKG} :

$C_{tx} = \text{Encrypt}_{PK_{DKG}}(\text{payload})$

Where C_{tx} is the threshold ciphertext. This ensures that no single node can decrypt the transaction until the threshold decryption ceremony occurs.

The payload includes:

- The Halo2 proof π (proving correct computation of δ_{tx})
- The nullifiers for all input notes
- The commitments for all output notes
- The encrypted notes (for Bob and Alice's change)
- The fee amount (public, as it's paid to validators)

Phase 2: PQ-Sphinx Onion Construction & Mixnet Routing

Business/Functional Process

Alice's wallet now needs to submit the transaction to the network without revealing her identity, IP address, transaction size, or timing to any observer. This is achieved through a 5-hop onion-routing mixer based on the Sphinx protocol, but upgraded with post-quantum cryptography (ML-KEM-768).

The purpose of this phase is to provide k -anonymity (anonymity set $> 1,000,000$) and traffic-analysis resistance, ensuring that even a global passive adversary cannot link Alice to her transaction. This is critical for NERV's privacy-by-default design.

Step 1: Relay Selection

Alice's wallet queries the network's on-chain relay registry, which contains a list of staked, attested relays. The registry is maintained on-chain and updated via governance votes. Relays are required to post a bond (stake) that can be slashed if they misbehave.

The wallet uses a Verifiable Random Function (VRF) to select 5 distinct relays from the registry:

```
seed = BLAKE3(wallet_private_key || current_block_height || nonce)
relays = select(relay_registry, seed, 5)
```

The relays are selected randomly but deterministically, ensuring that different wallets don't always pick the same relays.

Step 2: Ephemeral Key Generation

For each relay R_i (where $i = 1, 2, 3, 4, 5$), the wallet generates an ephemeral ML-KEM-768 public-private keypair:

```
(pk_i, sk_i) = MLKEM768.KeyGen()
```

These keys are used only for this transaction and are discarded afterward, providing forward secrecy.

Step 3: Layered Encryption (Onion Construction)

The wallet constructs the onion packet layer-by-layer, from the innermost layer to the outermost:

Innermost Layer (Layer 5):

The wallet encapsulates the transaction ciphertext C_{tx} for relay R_5 :

```
(ss_5, ct_5) = MLKEM768.Encapsulate(pk_5)
key_5 = BLAKE3.KDF(ss_5, domain="sphinx_layer_5")
Layer_5 = ChaCha20Poly1305.Encrypt(key_5, C_tx || nullifiers || commitments)
```

Layer 4:

The wallet wraps Layer 5 with routing information to R_5 :

```
(ss_4, ct_4) = MLKEM768.Encapsulate(pk_4)
key_4 = BLAKE3.KDF(ss_4, domain="sphinx_layer_4")
Layer_4 = ChaCha20Poly1305.Encrypt(key_4, R_5 || Layer_5 || padding)
```

Layer 3:

```
(ss_3, ct_3) = MLKEM768.Encapsulate(pk_3)
key_3 = BLAKE3.KDF(ss_3, domain="sphinx_layer_3")
Layer_3 = ChaCha20Poly1305.Encrypt(key_3, R_4 || Layer_4 || padding)
```

Layer 2:

```
(ss_2, ct_2) = MLKEM768.Encapsulate(pk_2)
key_2 = BLAKE3.KDF(ss_2, domain="sphinx_layer_2")
Layer_2 = ChaCha20Poly1305.Encrypt(key_2, R_3 || Layer_3 || padding)
```

Layer 1 (Outermost):

$(ss_1, ct_1) = \text{MLKEM768.Encapsulate}(pk_1)$

$key_1 = \text{BLAKE3.KDF}(ss_1, \text{domain}="sphinx_layer_1")$

$\text{Layer}_1 = \text{ChaCha20Poly1305.Encrypt}(key_1, R_2 \parallel \text{Layer}_2 \parallel \text{padding})$

The final packet contains ct_1 and Layer_1 , along with a Sphinx header that includes routing information but is designed to be indistinguishable from random data.

Step 4: Packet Padding & Sizing

To prevent size-analysis attacks, all packets are padded to a fixed size (2 KB). The padding is random data that will be stripped off by the final relay. The Sphinx format also includes a "dummy" tag that allows relays to identify and discard cover traffic without revealing which packets are real.

Step 5: Submission to First Relay

Alice's wallet broadcasts the packet to R_1 via libp2p's Kademlia DHT. The packet is sent using the Noise Protocol Framework with PQ-NOISE extensions (ML-KEM-768 + ChaCha20-Poly1305), ensuring that even the first relay cannot know the packet's origin.

Mathematical Formalism

Sphinx Packet Structure

The Sphinx packet consists of the following components:

1. Header: Contains the encrypted routing information for each hop
2. Payload: Contains the encrypted transaction data
3. MAC: Message authentication code to prevent tampering

The Sphinx protocol provides packet indistinguishability: the packet changes size and header at every hop, preventing timing and volume correlation attacks.

ML-KEM-768 Key Encapsulation

For each relay R_i , the wallet performs:

$(ss_i, ct_i) = \text{MLKEM768.Encapsulate}(pk_i)$

Where:

- pk_i is the relay's public key (posted on-chain)
- ss_i is the ephemeral shared secret
- ct_i is the ciphertext that the relay will use to recover ss_i

Symmetric Key Derivation

The symmetric key for each layer is derived using BLAKE3's Key Derivation Function (KDF):

$$\text{key}_i = \text{BLAKE3.KDF}(\text{ss}_i, \text{domain_separator} \parallel i)$$

The domain separator ensures that keys for different layers are distinct, even if the same ss_i were to be reused (which it isn't).

ChaCha20-Poly1305 Encryption

Each layer is encrypted using the AEAD cipher:

$$\text{Layer}_i = \text{ChaCha20Poly1305.Encrypt}(\text{key}_i, \text{nonce}_i, \text{plaintext}_i, \text{AD}_i)$$

Where:

- nonce_i is a unique 96-bit nonce (derived from ss_i or a counter)
- plaintext_i is the inner layer (including the next relay's address and the inner payload)
- AD_i is associated data (e.g., the Sphinx header, not encrypted but authenticated)

Relay Processing (Decapsulation)

When relay R_i receives the packet:

1. Key Decapsulation: The relay uses its secret key to recover the shared secret:
 $\text{ss}_i = \text{MLKEM768.Decapsulate}(\text{sk}_i, \text{ct}_i)$
2. Key Derivation: The relay derives the symmetric key:
 $\text{key}_i = \text{BLAKE3.KDF}(\text{ss}_i, \text{domain_separator} \parallel i)$
3. Decryption: The relay decrypts Layer_i :
 $\text{plaintext}_i = \text{ChaCha20Poly1305.Decrypt}(\text{key}_i, \text{nonce}_i, \text{Layer}_i, \text{AD}_i)$
4. Extraction: The relay extracts the next relay's address and the inner payload:
 $(R_{i+1}, \text{Layer}_{i+1}) = \text{plaintext}_i$
5. Forwarding: The relay forwards Layer_{i+1} to R_{i+1}

Timing Jitter & Cover Traffic

To defeat timing analysis, each relay applies a delay sampled from an exponential distribution:

$$\Delta t \sim \text{Exp}(\mu = 100\text{ms}, \sigma = 200\text{ms})$$

The probability density function is: $f(\Delta t) = (1/\sigma) \cdot e^{(-\Delta t/\sigma)}$

Relays also generate dummy packets (cover traffic) using a local NWO Perceptron that has learned to mimic real traffic distributions. The Perceptron is trained to predict the next packet's size, timing, and destination based on recent network activity. This ensures that real and dummy packets are indistinguishable to an observer.

Dummy Packet Generation

The local NWO Perceptron for relay R_i computes:

$$\hat{y}_{\text{dummy}} = W_{\text{dummy}} \cdot X_{\text{traffic}} + b_{\text{dummy}}$$

Where:

- W_{dummy} is the relay's local weight matrix (updated via gradient descent)
- X_{traffic} are the features of recent traffic (size, timing, destination entropy)
- b_{dummy} is the bias term

The output $\hat{y}_{\text{dummy}}$ determines the dummy packet's properties (size, timing, destination). The Perceptron is updated in real-time using the Adam optimizer to minimize the difference between predicted and actual traffic distributions.

Traffic Analysis Resistance

The combination of:

1. 5-hop onion routing (no single relay knows both source and destination)
2. ML-KEM-768 post-quantum encryption (immune to Shor's algorithm)
3. Timing jitter (exponential delays)
4. Cover traffic (dummy packets)
5. Packet indistinguishability (fixed sizes, randomized headers)

Provides formal k-anonymity. The ProVerif formal verification model proves:

- k-anonymity > 1,000,000 against passive global adversary
- k-anonymity > 100,000 against active adversary controlling < 33% of relays

Phase 3: Mempool Integration & Gossip

Business/Functional Process

After traversing the 5-hop mixnet, the final relay R_5 (the exit node) receives the innermost layer containing the transaction ciphertext C_tx. At this point, R_5 does not decrypt C_tx—it simply passes it to a validator node that is responsible for maintaining the mempool.

The transaction now enters the "Threshold Decrypted Mempool," which is a pool of encrypted transactions that no single node can read. This is a significant departure from traditional blockchains where the mempool is fully transparent and can be analyzed by MEV bots, front-runners, and privacy attackers.

Step 1: Relay-to-Validator Handoff

The final relay R_5 forwards C_tx to a randomly selected validator node via a secure, authenticated channel. The validator's identity is known from the on-chain validator registry.

Step 2: Mempool Acceptance

The validator performs a series of checks on C_tx:

1. Format Validation: Verify that the ciphertext has the expected structure and size
2. Fee Check: Ensure the transaction fee is sufficient (the fee amount is included in the plaintext, but encrypted; the validator checks the fee commitment)
3. Anti-Spam Check: Verify that the transaction hasn't been submitted too many times (using a rate-limiting mechanism)
4. Proof-of-Work (Optional): If the network is under spam attack, a small PoW may be required

If all checks pass, the validator adds C_tx to the encrypted mempool.

Step 3: Mempool Gossip

The validator gossips C_tx to other validators using GossipSub (a libp2p-based pubsub protocol). GossipSub ensures that all validators in the network eventually receive a copy of C_tx with high probability and low latency.

The gossip process is:

1. The receiving validator verifies C_tx hasn't been seen before (using a bloom filter or seen-set)
2. The validator forwards C_tx to a random subset of its peers (typically ~6 peers)
3. Each peer repeats the process, causing the transaction to spread through the network rapidly

Step 4: Mempool Batching

Validators collect encrypted transactions and batch them for block production. The batch size is dynamic but typically contains up to 10,000 transactions per batch to optimize throughput and reduce per-transaction overhead.

The batch $B = \{C_{tx}^1, C_{tx}^2, \dots, C_{tx}^n\}$ is maintained in memory and periodically committed to the block.

Step 5: Mempool Persistence (Optional)

In case of a validator crash or network partition, the mempool may be persisted to disk to prevent transaction loss. However, the encrypted nature of the mempool means that even if the disk is compromised, the transaction contents remain confidential.

Mathematical Formalism

Mempool State Representation

The mempool can be represented as a set of encrypted transactions:

$$\text{Mempool} = \{C_{tx}^1, C_{tx}^2, \dots, C_{tx}^n\}$$

Where each C_{tx}^i is: $C_{tx}^i = \text{Encrypt_PK_DKG}(\text{payload}^i)$

And payload^i contains:

- The Halo2 proof π^i
- The nullifiers $N^i = \{\text{nullifier}_1, \text{nullifier}_2, \dots, \text{nullifier}_m\}$
- The commitments $C^i = \{\text{commitment}_1, \text{commitment}_2, \dots, \text{commitment}_k\}$
- The encrypted notes (for recipients)
- The fee fee^i

Deduplication

To prevent double-spending, validators maintain a seen-set of nullifiers:

$$\text{SeenNullifiers} = \{\text{nullifier} : \text{nullifier has been seen in mempool or on-chain}\}$$

When a new transaction C_{tx} is received, the validator cannot check the nullifiers directly (they're encrypted). Instead, the validator relies on the ZK proof π to assert that the nullifiers are unique. The proof includes a commitment to the nullifiers, which the validator can check against the seen-set.

The seen-set is a Merkle tree of nullifiers, allowing efficient membership proofs:

$\text{root} = \text{MerkleRoot}(\text{SeenNullifiers})$

The ZK proof π includes a proof that the nullifiers are not in the seen-set:

$\pi_{\text{nullifier}} = \text{Prove}(\text{nullifiers} \notin \text{SeenNullifiers})$

Batching Efficiency

By batching transactions, the network achieves economies of scale. The overhead of the ZK proof is amortized across 10,000 transactions, resulting in a per-transaction proof size of:

$\text{ProofSize}_{\text{per-tx}} \approx (400 \text{ bytes}) / (10,000) = 0.04 \text{ bytes/tx}$

The mempool size for 10,000 transactions is approximately:

$\text{MempoolSize} = 10,000 \times (1,088 \text{ bytes for ML-KEM ciphertext} + 400 \text{ bytes proof}) \approx 14.88 \text{ MB}$

This is manageable for modern validators with 64GB+ RAM.

GossipSub Protocol

GossipSub is a mesh-based pubsub protocol with the following properties:

1. Mesh Topology: Each validator maintains a mesh of ~6 peers (configurable)
2. Gossip Heartbeat: Validators send I HAVE messages to their peers, announcing which messages they have
3. I HAVE/I WANT: Peers request messages they don't have via I WANT messages
4. GRAFT/PRUNE: Validators dynamically adjust their mesh to optimize connectivity

The protocol ensures high message dissemination speed (< 200ms for 10,000 nodes) and low overhead.

Mempool Metrics

Validators track mempool metrics for load prediction:

Metric	Description	Typical Value

Mempool Size	Number of transactions in mempool	10,000 - 1,000,000
Mempool Growth Rate	Rate of incoming transactions	1,000 - 100,000 txs/sec
Queue Depth	Depth of pending transactions per account	0 - 100
Fee Distribution	Histogram of transaction fees	Skewed (many low, few high)
Age of Oldest Tx	Latency of mempool clearing	< 5 seconds

These metrics are used by the NWO Perceptron to predict load and trigger dynamic sharding.

Phase 4: Threshold Decryption Ceremony

Business/Functional Process

The threshold decryption ceremony is the moment when the encrypted mempool transactions are finally decrypted and executed. This is a critical security event because it's the first time the transaction contents become visible to any node. To maintain privacy, the ceremony is designed to prevent any single node from seeing all transactions and to ensure that decrypted data is immediately zeroed out after execution.

Step 1: Block Proposal

A validator (the block producer) is selected via a stake-weighted round-robin or VRF-based mechanism. The block producer is responsible for:

1. Selecting a batch of encrypted transactions from the mempool
2. Coordinating the threshold decryption ceremony
3. Assembling the block
4. Proposing the block to the network

The block producer is incentivized to include as many transactions as possible (to maximize fees) but must avoid exceeding the block size limit or causing excessive latency.

Step 2: Partial Decryption Generation

For each encrypted transaction C_{tx} in the batch, the block producer requests partial decryptions from the validator set.

Validator V_i has a DKG secret share s_i (a scalar in the BLS12-381 field). For each C_{tx} , V_i computes:

$$P_i = C_{tx}^{(s_i)}$$

Where C_{tx} is treated as a group element in the BLS12-381 G_1 group. This is a partial decryption—it reveals nothing about the plaintext on its own.

Step 3: Partial Decryption Collection

The block producer collects partial decryptions from t validators, where t is the threshold (e.g., $t = \text{ceil}(2n/3)$ for a 2/3 quorum). Each partial decryption P_i is accompanied by a proof of correct decryption (to prevent malicious validators from submitting invalid shares).

The proof ensures that P_i was indeed computed as $C_{tx}^{(s_i)}$ using the correct secret share.

Step 4: Lagrange Interpolation

Once t partial decryptions are collected, the block producer combines them using Lagrange interpolation to recover the full decryption:

$$\text{SharedSecretPoint} = \prod_{i=1}^t P_i^{(\lambda_i)} = C_{tx}^{(\sum s_i \lambda_i)} = C_{tx}^s$$

Where:

- P_i are the partial decryptions
- λ_i are the Lagrange coefficients
- s is the full DKG secret (never revealed to any single node)

Step 5: Decryption

The block producer derives the symmetric key from the shared secret point:

$K = \text{BLAKE3.KDF}(\text{SharedSecretPoint}, \text{domain_separator})$

And decrypts the transaction payload:

$\text{payload} = \text{ChaCha20Poly1305.Decrypt}(K, C_{\text{tx}})$

Step 6: Immediate Zeroing

After decryption, the transaction payload is executed (Phase 5), and then the plaintext is immediately zeroed out from memory. The block producer uses a secure memory clearing routine to prevent any residual data from being leaked (e.g., via memory dumps, cold boot attacks, or side channels).

Step 7: Batch Decryption

The process is repeated for all transactions in the batch. Once all transactions are decrypted and executed, the block producer finalizes the block.

Mathematical Formalism

Distributed Key Generation (DKG) Recap

The DKG protocol (based on Feldman VSS) produces:

1. A collective public key PK_{DKG} (known to all)
2. Secret shares s_i for each validator V_i (known only to V_i)
3. Public verification keys $v_i = g^{(s_i)}$ for each validator

The threshold t is chosen as $t = \text{ceil}(2n/3)$ for a Byzantine fault-tolerant system.

Feldman VSS Parameters

For a group of n validators with threshold t :

1. Each validator V_i generates a random polynomial:

$$f_i(x) = a_{\{i,0\}} + a_{\{i,1\}}x + a_{\{i,2\}}x^2 + \dots + a_{\{i,t-1\}}x^{(t-1)}$$
 Where $a_{\{i,0\}} = s_i$ is the validator's secret.

2. Each validator broadcasts commitments:
 $C_{i,j} = g^{(a_{i,j})}$ for $j = 0, 1, \dots, t-1$
3. Each validator V_j receives shares $f_i(j)$ from validator V_i and verifies:
 $g^{(f_i(j))} = \prod_{k=0}^{t-1} (C_{i,k})^{(j^k)}$
4. The collective public key is:
 $PK_DKG = \prod_{i=1}^n C_{i,0} = g^{(\sum s_i)} = g^s$
5. Each validator's final secret share is:
 $s_j = \sum_{i=1}^n f_i(j)$

Threshold Decryption

For a ciphertext C_{tx} (a group element in G_1):

1. Validator V_i computes partial decryption:
 $P_i = C_{tx}^{(s_i)}$
2. The block producer collects t partial decryptions and computes:
 $SharedSecretPoint = \prod_{i \in S} P_i^{(\lambda_i)}$
Where S is the set of t validators and λ_i are Lagrange coefficients:
 $\lambda_i = \prod_{j \in S, j \neq i} j / (j - i)$
3. The symmetric key is derived:
 $K = \text{BLAKE3.KDF}(SharedSecretPoint, "mempool_decryption")$
4. The payload is decrypted:
 $payload = \text{ChaCha20Poly1305.Decrypt}(K, \text{nonce}, C_{tx}, AD)$

Proof of Correct Decryption

To prevent malicious validators from submitting invalid partial decryptions, each validator must provide a proof of correct decryption. This proof is a zero-knowledge proof that $P_i = C_{tx}^{(s_i)}$ without revealing s_i .

The proof is based on the BLS12-381 pairing:

1. Validator V_i knows s_i and has public verification key $v_i = g^{(s_i)}$
2. The validator computes $P_i = C_{tx}^{(s_i)}$
3. The proof is: $\pi_i = \text{Prove}(e(C_{tx}, v_i) = e(P_i, g))$
 Where e is the BLS12-381 pairing function.

The pairing equation holds iff $P_i = C_{tx}^{(s_i)}$. This proof can be verified publicly by any node.

Security Properties

The threshold decryption scheme provides:

1. No Single Point of Failure: No single validator can decrypt transactions on their own (requires t validators)
 2. Malicious Validator Resistance: Invalid partial decryptions are detected (via proof verification) and the offending validator is slashed
 3. Forward Secrecy: Even if a validator's secret share is compromised, past transactions remain private (the shared secret is ephemeral)
 4. Post-Quantum Security: The underlying cryptographic primitives (ML-KEM-768, Dilithium-3, BLAKE3) are post-quantum secure
-

Phase 5: State Transition & NWO Execution

Business/Functional Process

Once the mempool transactions are decrypted, the block producer executes them against the current state. In NERV V2.0, the state is represented as a 512-byte embedding vector (64 dimensions of 64-bit fixed-point numbers). The state transition is linear and perfectly homomorphic, enabling extremely fast execution.

Step 1: Delta Aggregation

The block producer collects the homomorphic deltas δ_{tx^i} from each decrypted transaction. Each δ_{tx^i} is a 64-dimensional vector that represents the change to the state embedding caused by that transaction.

Due to the linearity of the NWO Perceptron, the deltas can be aggregated simply by vector addition:

$$\Delta_B = \sum_{i=1}^n \delta_{tx^i}$$

Where n is the number of transactions in the batch (up to 10,000).

Step 2: Embedding Update

The canonical state embedding e_t is updated via exact vector addition:

$$e_{t+1} = e_t + \Delta_B$$

This is a constant-time operation (64 additions) regardless of the number of transactions in the batch. This is the key scalability breakthrough: the state update is $O(1)$ in the number of transactions, enabling >1M TPS.

Step 3: Nullifier Publication

The nullifiers from all transactions in the batch are added to the seen-set (on-chain). This prevents double-spending: any future transaction attempting to use a spent note will be rejected because its nullifier is already in the seen-set.

The nullifier Merkle tree is updated with the new nullifiers:

$$\text{root}_{\{t+1\}} = \text{MerkleRoot}(\text{root}_t \cup \text{Nullifiers_batch})$$

Step 4: Commitment Publication

The commitments for the new output notes are added to the commitment tree. This allows future transactions to spend these notes (the spender must know the note's secret key to generate the correct nullifier).

$$\text{commitment_root}_{\{t+1\}} = \text{MerkleRoot}(\text{commitment_root}_t \cup \text{Commitments_batch})$$

Step 5: Fee Distribution

The transaction fees are distributed to validators (block producer and voting validators) according to the consensus rules. The fee is taken from the transaction (inputs - outputs = fee) and added to the validators' accounts.

Step 6: State Root Computation

The new state root is computed as:

$$h_{\{t+1\}} = \text{BLAKE3}(\text{e}_{\{t+1\}} \parallel \text{nullifier_root}_{\{t+1\}} \parallel \text{commitment_root}_{\{t+1\}} \parallel \text{validator_set}_{\{t+1\}})$$

This root is committed to the block header and serves as the canonical state identifier for this block height.

Mathematical Formalism

State Representation

The full state at height t is:

$$S_t = (\text{e}_t, \text{nullifier_root}_t, \text{commitment_root}_t, \text{validator_set}_t)$$

Where:

- $e_t \in \mathbb{R}^{64}$ is the embedding vector (512 bytes)
- $nullifier_root_t$ is the Merkle root of the nullifier tree
- $commitment_root_t$ is the Merkle root of the commitment tree
- $validator_set_t$ is the set of active validators (with their stakes and public keys)

Homomorphic Delta Computation (Client-Side)

For a transaction tx , the client computes:

$$\delta_{tx} = W \cdot \Delta S_{tx} + b_{tx}$$

Where:

- $W \in \mathbb{R}^{(64 \times N)}$ is the network's weight matrix
- $\Delta S_{tx} \in \mathbb{R}^N$ is the transaction's state delta vector
- $b_{tx} \in \mathbb{R}^{64}$ is the transaction-specific bias

The client provides a ZK proof π_{tx} that proves the correctness of this computation without revealing ΔS_{tx} or b_{tx} .

Delta Aggregation (Validator-Side)

For a batch of n transactions $\{tx_1, tx_2, \dots, tx_n\}$, the validator aggregates:

$$\Delta_B = \sum_{i=1}^n \delta_{tx_i}$$

Due to the linearity of the NWO Perceptron, the aggregation is exact:

$$\Delta_B = \sum_{i=1}^n (W \cdot \Delta S_{tx_i} + b_{tx_i}) = W \cdot \sum_{i=1}^n \Delta S_{tx_i} + \sum_{i=1}^n b_{tx_i}$$

Embedding Update

The state embedding is updated:

$$e_{t+1} = e_t + \Delta_B$$

Nullifier Tree Update

The nullifier tree is updated with the new nullifiers $\{nullifier_1, \dots, nullifier_m\}$:

$$nullifier_root_{t+1} = \text{MerkleRoot}(nullifier_root_t, nullifier_1, \dots, nullifier_m)$$

Commitment Tree Update

The commitment tree is updated with the new commitments $\{commitment_1, \dots, commitment_k\}$:

$\text{commitment_root}_{\{t+1\}} = \text{MerkleRoot}(\text{commitment_root}_t, \text{commitment}_1, \dots, \text{commitment}_k)$

Validator Set Update

The validator set may change if:

1. A validator is slashed (stake reduced)
2. A new validator is added (stake deposited)
3. A validator withdraws (stake removed)

The validator set is updated deterministically based on the on-chain staking contract.

State Root Computation

The final state root is:

$h_{\{t+1\}} = \text{BLAKE3}(e_{\{t+1\}} \parallel \text{nullifier_root}_{\{t+1\}} \parallel \text{commitment_root}_{\{t+1\}} \parallel \text{validator_set}_{\{t+1\}})$

This root is included in the block header and is used for light-client verification.

Phase 6: Self-Evolution (Adam & Huber Loss)

Business/Functional Process

The self-evolution phase is what makes NERV a "living" blockchain. Unlike traditional blockchains where the protocol is static, NERV continuously learns and adapts to changing transaction patterns. The network's weights W and biases b are updated every block using the Adam optimizer and Huber loss.

The purpose of this phase is:

1. Adaptation: The network learns to represent the state more efficiently over time
2. Optimization: The network's compression improves, reducing the need for sharding
3. Intelligence: The network becomes more accurate at predicting future states
4. Security: The network can detect and respond to anomalous patterns

Step 1: Prediction Computation

After the state transition, validators compute the prediction $\hat{y}$ of what the embedding should be, based on the old embedding e_t and the aggregated delta Δ_B :

$$\hat{y} = W \cdot e_t + W \cdot \Delta_B + b$$

This is the Perceptron's "guess" of what the new embedding will be.

Step 2: Target Measurement

The actual new embedding e_{t+1} is known from the state transition. This is the "ground truth" target y that the Perceptron should have predicted.

Step 3: Huber Loss Calculation

The Huber loss is computed to measure the error between the prediction and the target. Huber loss is chosen because it's robust to outliers—unlike MSE, it doesn't overreact to extreme values.

The Huber loss is:

$$L_\delta(y, \hat{y}) = \{ \frac{1}{2}(y - \hat{y})^2 \text{ if } |y - \hat{y}| \leq \delta ; \delta \cdot |y - \hat{y}| - \frac{1}{2}\delta^2 \text{ otherwise } \}$$

Where δ is a hyperparameter (typically 1.0). For small errors, it's quadratic (like MSE); for large errors, it's linear (like MAE).

Step 4: Gradient Computation

Validators compute the gradient of the loss with respect to the weights:

$$g = \nabla_W L_\delta(y, \hat{y})$$

The gradient for a linear Perceptron is:

$$g = \partial L_\delta / \partial W = (\partial L_\delta / \partial \hat{y}) \cdot (\partial \hat{y} / \partial W) = (\partial L_\delta / \partial \hat{y}) \cdot x$$

Where $x = e_t + \Delta_B$ is the input to the Perceptron.

Step 5: Gradient Clipping

To prevent adversarial updates (e.g., a malicious validator submitting an extreme gradient), the gradient is clipped to a maximum norm:

$$g_{\text{clipped}} = \{ g \text{ if } \|g\|_2 \leq \text{clip_norm} ; \text{clip_norm} \cdot (g / \|g\|_2) \text{ otherwise } \}$$

Step 6: Adam Optimizer Update

Validators update the first and second moments of the gradient:

$$m = \beta_1 \cdot m + (1 - \beta_1) \cdot g_clipped$$

$$v = \beta_2 \cdot v + (1 - \beta_2) \cdot g_clipped^2$$

Bias correction is applied:

$$m_hat = m / (1 - \beta_1^t)$$

$$v_hat = v / (1 - \beta_2^t)$$

The new weights are:

$$W_new = W - \alpha \cdot (m_hat / (\sqrt{v_hat} + \epsilon))$$

Step 7: Consensus on New Weights

Validators broadcast the hash of their computed W_new in their block votes. If $\geq 67\%$ of validators agree on the same W_new (within a small tolerance), the network officially adopts the new weights for the next block.

Step 8: Useful-Work Rewards

Validators who submitted gradients that successfully reduced the loss (i.e., their W_new matched the consensus) are eligible for Useful-Work rewards. The reward is proportional to the validator's contribution to the loss reduction, incentivizing high-quality gradient computation.

Mathematical Formalism

Perceptron Forward Pass

The Perceptron computes:

$$\hat{y} = W \cdot x + b$$

Where:

- $W \in \mathbb{R}^{(64 \times N)}$ is the weight matrix
- $x \in \mathbb{R}^N$ is the input vector (features from the state)
- $b \in \mathbb{R}^{64}$ is the bias vector
- $\hat{y} \in \mathbb{R}^{64}$ is the prediction

Huber Loss

For a target $y \in \mathbb{R}^{64}$ and prediction $\hat{y} \in \mathbb{R}^{64}$:

$$L_\delta(y, \hat{y}) = \sum_{j=1}^{64} L_\delta(y_j, \hat{y}_j)$$

Where:

$$L_{\delta}(y_j, \hat{y}_j) = \{ \frac{1}{2}(y_j - \hat{y}_j)^2 \text{ if } |y_j - \hat{y}_j| \leq \delta ; \delta \cdot |y_j - \hat{y}_j| - \frac{1}{2}\delta^2 \text{ otherwise } \}$$

Gradient Computation

The gradient of the Huber loss with respect to the prediction is:

$$\partial L_{\delta} / \partial \hat{y}_j = \{ \hat{y}_j - y_j \text{ if } |y_j - \hat{y}_j| \leq \delta ; -\delta \cdot \text{sign}(y_j - \hat{y}_j) \text{ otherwise } \}$$

The gradient with respect to the weights is:

$$g_{\{j,k\}} = (\partial L_{\delta} / \partial \hat{y}_j) \cdot x_k$$

Where:

- j indexes the output dimension (1 to 64)
- k indexes the input feature (1 to N)
- x_k is the k -th input feature

Adam Optimizer Update

The Adam optimizer parameters:

- $\beta_1 = 0.9$ (decay rate for first moment estimate)
- $\beta_2 = 0.999$ (decay rate for second moment estimate)
- $\alpha = 0.001$ (learning rate)
- $\epsilon = 10^{-8}$ (small constant for numerical stability)

The update equations:

$$\text{First moment estimate: } m_{\{j,k\}} \leftarrow \beta_1 \cdot m_{\{j,k\}} + (1 - \beta_1) \cdot g_{\{j,k\}}$$

$$\text{Second moment estimate: } v_{\{j,k\}} \leftarrow \beta_2 \cdot v_{\{j,k\}} + (1 - \beta_2) \cdot g_{\{j,k\}}^2$$

$$\text{Bias correction: } m_{\text{hat}}_{\{j,k\}} = m_{\{j,k\}} / (1 - \beta_1^t) ; v_{\text{hat}}_{\{j,k\}} = v_{\{j,k\}} / (1 - \beta_2^t)$$

$$\text{Weight update: } W_{\text{new}}_{\{j,k\}} = W_{\{j,k\}} - \alpha \cdot (m_{\text{hat}}_{\{j,k\}} / (\sqrt{v_{\text{hat}}_{\{j,k\}}} + \epsilon))$$

Consensus on Weights

Validators compute W_{new} and broadcast:

$$h_W = \text{BLAKE3}(W_{\text{new}})$$

Consensus is reached if:

$$\sum \{v: h_{W,v} = h_{W,\text{majority}}\} \text{ stake}_v \geq 0.67 \times \sum \{v \in V\} \text{ stake}_v$$

Where V is the validator set and $h_{W,\text{majority}}$ is the hash that receives the most stake-weighted votes.

Useful-Work Reward

The reward for validator v is proportional to the loss reduction they contributed:

$$\text{reward}_v = \text{pool} \times (\text{loss_reduction}_v / \sum_i \text{loss_reduction}_i)$$

Where:

- pool is the total useful-work reward pool (from emissions and fees)
- $\text{loss_reduction}_v = L_\delta(y, \hat{y}_{\text{old}}) - L_\delta(y, \hat{y}_{\text{new},v})$
- $\hat{y}_{\text{new},v}$ is the prediction using $W_{\text{new},v}$ (validator's proposed weights)

Validators who submit invalid gradients (e.g., causing loss to increase) are penalized (slashed).

The following section details the four core equations of the Adam optimizer used in the self-evolution phase.

1. First Moment Estimate (Momentum)

$$\text{Equation: } m_{\{j,k\}} \leftarrow \beta_1 \cdot m_{\{j,k\}} + (1 - \beta_1) \cdot g_{\{j,k\}}$$

- Meaning: This calculates the exponential moving average of past gradients. It represents the "momentum" of the weight update, smoothing out noisy gradients to maintain a consistent direction.
- Computation from Huber Loss:
 - $g_{\{j,k\}}$ is the gradient computed directly from the Huber loss: $g_{\{j,k\}} = (\partial L_\delta / \partial \hat{y}_j) \cdot x_k$.
 - If the Huber loss gradient is consistent (e.g., always positive for a specific weight), $m_{\{j,k\}}$ grows, accelerating the update.
 - If the gradient oscillates (positive then negative), $m_{\{j,k\}}$ stays near zero, preventing erratic updates.
- Parameters: $\beta_1 = 0.9$ means the estimate retains 90% of the past momentum and adds 10% of the new gradient.

2. Second Moment Estimate (Adaptive Scaling)

Equation: $v_{\{j,k\}} \leftarrow \beta_2 \cdot v_{\{j,k\}} + (1 - \beta_2) \cdot g_{\{j,k\}}^2$

- Meaning: This calculates the exponential moving average of the squared gradients. It represents the uncentered variance (magnitude) of the gradients.
- Computation from Huber Loss:
 - It squares the Huber gradient $g_{\{j,k\}}$. Large gradients (outliers) result in a large $v_{\{j,k\}}$; small gradients result in a small $v_{\{j,k\}}$.
 - Purpose: This term adapts the learning rate per parameter. If a weight has historically large gradients (steep slope), $v_{\{j,k\}}$ becomes large, which will reduce the effective step size in the final update to prevent overshooting.
- Parameters: $\beta_2 = 0.999$ means this average is over a longer history than the first moment.

3. Bias Correction

Equations: $m_{\text{hat}}_{\{j,k\}} = m_{\{j,k\}} / (1 - \beta_1^t)$ $v_{\text{hat}}_{\{j,k\}} = v_{\{j,k\}} / (1 - \beta_2^t)$

- Meaning: This corrects the initialization bias. Since m and v are initialized to zero, their early estimates (at low block height t) are artificially small, biased toward zero.
- Computation:
 - At block $t=1$, $1 - \beta_1^1 = 0.1$. Dividing by 0.1 scales the small initial estimate up by 10x, making it an unbiased estimate of the true gradient.
 - As t (block height) increases, β^t approaches 0, the denominator approaches 1, and the correction becomes negligible.
- Why it matters in NERV: Without this, the network's weights would update too slowly in the first few blocks, delaying the "learning" of optimal state representations during critical early network phases.

4. Weight Update

Equation: $W_{\text{new}}_{\{j,k\}} = W_{\{j,k\}} - \alpha \cdot (m_{\text{hat}}_{\{j,k\}} / (\sqrt{v_{\text{hat}}_{\{j,k\}}} + \epsilon))$

- Meaning: This is the final step that applies the calculated changes to the global weight matrix W .
- How it works:
 - Direction: Determined by $m_hat_{\{j,k\}}$ (the bias-corrected momentum).
 - Step Size: Adaptively scaled by $1 / (\sqrt{v_hat_{\{j,k\}}})$.
 - If $v_hat_{\{j,k\}}$ is large (volatile/large gradients), the denominator is large, and the step size shrinks.
 - If $v_hat_{\{j,k\}}$ is small (stable/small gradients), the denominator is small, and the step size increases.
 - Learning Rate (α): The global base step size (0.001).
 - Stability (ϵ): A tiny constant (10^{-8}) prevents division by zero if $v_hat_{\{j,k\}}$ is zero.
- Result: The weight $W_{\{j,k\}}$ is nudged in the direction that minimizes the Huber loss, with a step size automatically tuned for that specific weight's history.

Initialization of the First Moment Estimate ($m_{\{j,k\}}$)

In the very first iteration (block $t=1$), the first moment estimate $m_{\{j,k\}}$ is initialized to 0.

Initialization Process

1. Genesis State: When the NERV network launches (or when the Adam optimizer state is reset), all moment estimates ($m_{\{j,k\}}$ and $v_{\{j,k\}}$) are set to zero for every weight in the matrix W .
2. First Update ($t=1$):
 - The Huber loss is calculated, yielding the first gradient $g_{\{j,k\}}$.
 - The equation runs: $m_{\{j,k\}} \leftarrow 0.9 \cdot 0 + (1 - 0.9) \cdot g_{\{j,k\}}$.
 - Result: $m_{\{j,k\}} = 0.1 \cdot g_{\{j,k\}}$.
3. Bias Correction Role: Because the raw value ($0.1 \cdot g_{\{j,k\}}$) is much smaller than the actual gradient, the Bias Correction step is critical here. It divides by $(1 - \beta_1^1) = 0.1$, effectively scaling the value back up:
 - $m_hat_{\{j,k\}} = (0.1 \cdot g_{\{j,k\}}) / 0.1 = g_{\{j,k\}}$.
 - This ensures the first update step uses the full gradient magnitude, preventing the network from "learning" too slowly at the start.

Summary

$m_{\{j,k\}}$ starts at 0, and the math (specifically the bias correction) ensures this zero-initialization does not hinder the first update.

Phase 7: AI-Native Consensus & Finality

Business/Functional Process

NERV's consensus mechanism is "AI-Native" because it uses the NWO Perceptron itself for fast prediction and finality. Validators predict the next embedding hash using the Perceptron, and if a quorum agrees, the block is finalized probabilistically. This is significantly faster than traditional BFT consensus (600ms vs. 2-5 seconds).

Step 1: Perceptron Hash Prediction

After the state transition (Phase 5) and the weight update (Phase 6), validators use the updated Perceptron to predict the next block's embedding hash.

The Perceptron is trained to predict the next embedding from the current embedding and the incoming deltas. So:

$$e_{\text{pred}} = W_{\text{new}} \cdot (e_t + \Delta_B) + b = W_{\text{new}} \cdot e_{\{t+1\}} + b$$

This is the Perceptron's prediction of the embedding after the *next* batch. The hash of this prediction is $h_{\text{pred}} = \text{BLAKE3}(e_{\text{pred}})$.

Step 2: Vote Generation

Validators generate BLS signatures on h_{pred} and broadcast them:

$$\text{vote}_v = \text{BLS.Sign}(\text{sk}_v, h_{\text{pred}})$$

Where sk_v is validator v 's BLS private key. The vote also includes the validator's current reputation score (updated via the federated learning process).

Step 3: Vote Aggregation & Quorum Check

The block producer collects votes from validators and checks if a quorum is reached. The quorum is weighted by stake $\times$ reputation:

$$\text{weight}_v = \text{stake}_v \times \text{reputation}_v$$

Quorum is reached if:

$$\sum\{v: h_{\text{pred},v} = h_{\text{pred},\text{majority}}\} \text{weight}_v \geq 0.67 \times \sum\{v \in V\} \text{weight}_v$$

If quorum is reached, the block is considered probabilistically finalized. This gives instant finality in $\sim 600\text{ms}$.

Step 4: Challenge Phase (Dispute Resolution)

If quorum is not reached (because validators disagree on the predicted hash), a challenge phase is triggered. This happens in $< 0.01\%$ of blocks.

A validator may post a bonded challenge (1-5% of their stake) if they believe the majority prediction is wrong. The challenge triggers a Monte Carlo simulation:

1. 32 random validators are selected (via VRF)
2. Each runs 10,000 parallel simulations of the state transition using random subsets of the batch
3. Each validator independently computes the correct embedding
4. The majority outcome wins

The losing side is slashed (0.5-5% of their stake), and the challenger's bond is either returned (if they were right) or forfeited (if they were wrong).

Step 5: Cryptographic Finality

After the challenge phase (or immediately if no challenge), the BLS signatures are aggregated into a single 48-byte signature:

$$\text{agg_sig} = \text{BLS.Aggregate}(\text{votes})$$

This aggregated signature provides cryptographic finality: it proves that a supermajority of validators agreed on the block.

Step 6: Block Commitment

The block is committed to the chain. The block header contains:

- The new state root h_{t+1}

- The aggregated BLS signature agg_sig
- The list of transactions in the block (or their hashes)
- The new weights W_{new} (or a commitment to them)

Mathematical Formalism

Perceptron Prediction

For a batch B with aggregated delta Δ_B , the Perceptron predicts the next embedding:

$$e_{\text{pred}} = W_{\text{new}} \cdot (e_t + \Delta_B) + b$$

The predicted hash is: $h_{\text{pred}} = \text{BLAKE3}(e_{\text{pred}})$

Vote Structure

A validator's vote is:

$$\text{vote}_v = (h_{\text{pred}}, \sigma_v, \text{reputation}_v)$$

Where:

- h_{pred} is the predicted hash
- $\sigma_v = \text{BLS.Sign}(\text{sk}_v, h_{\text{pred}})$ is the BLS signature
- reputation_v is the validator's reputation score (updated via federated learning)

Quorum Check

The weighted quorum is:

$$Q = (\sum_{v \in V_{\text{agree}}} \text{stake}_v \times \text{reputation}_v) / (\sum_{v \in V} \text{stake}_v \times \text{reputation}_v)$$

Quorum is reached if $Q \geq 0.67$.

Monte Carlo Dispute Resolution

The challenge phase uses Monte Carlo simulation:

1. Select 32 validators via VRF:
 $\text{selector} = \text{VRF}(\text{sk}_{\text{block_producer}}, \text{block_height}, \text{challenge_seed})$
2. For each of the 32 validators, run 10,000 simulations:
 $e_{\text{sim}}^{(k)} = e_t + \sum_{i \in S_k} \delta_{\text{tx}_i}$
 Where S_k is a random subset of the batch.
3. Each validator computes the majority outcome and votes.
4. The final outcome is determined by majority vote among the 32 validators.

BLS Signature Aggregation

The BLS signatures are aggregated:

$$\text{agg_sig} = \sum_{v \in V_{\text{agree}}} \sigma_v$$

This is verified using the aggregated public key:

$$\text{pk_agg} = \sum_{v \in V_{\text{agree}}} \text{pk}_v$$

The verification equation:

$$e(\text{agg_sig}, g) = e(\text{pk_agg}, h_{\text{pred}})$$

Where e is the BLS12-381 pairing function.

Finality Properties

NERV provides:

1. Probabilistic Finality: ~600ms (quorum reached)
2. Economic Finality: ~1.8s (challenge phase + cryptographic finality)
3. Safe Under Partial Synchrony: BFT + challenge mechanism ensures safety
4. Post-Quantum Security: BLS is used for liveness only; long-term security relies on Dilithium-3

Phase 8: Dynamic Sharding & Erasure Coding

Business/Functional Process

Dynamic sharding is NERV's mechanism for infinite horizontal scalability. Shards automatically split and merge based on load, ensuring that the network can handle any amount of traffic without manual intervention.

Step 1: Load Monitoring

A secondary NWO Perceptron (the "sharding Perceptron") monitors network load in real-time. It ingests features such as:

- TPS (transactions per second) per shard
- Mempool size (pending transactions)

- Inter-shard message queue depth
- Gas limit per block
- Cross-shard transaction ratio

Step 2: Load Prediction

The sharding Perceptron predicts the load for the next 15 seconds:

$$p = \text{Sigmoid}(W_{\text{shard}} \cdot X + b_{\text{shard}})$$

Where $p \in [0, 1]$ is the probability of overload. If $p > 0.90$, a shard split is triggered.

Step 3: Split Proposal

A validator detects the overload probability and posts a bonded SplitProposal:

$$\text{proposal} = (\text{shard_id}, \text{height}, \text{seed}, \text{bond})$$

The seed is derived deterministically:

$$\text{seed} = \text{BLAKE3}(\text{shard_id} \parallel \text{height} \parallel \text{chain_constant})$$

Step 4: Split Approval

Validators vote on the split proposal. If $\geq 67\%$ of the shard's stake approves, the split proceeds.

Step 5: Embedding Bisection

The current embedding e_t (512 bytes) is deterministically bisected into two child embeddings:

$$e_{\text{left}} = (e_t - \text{proj}_d(e_t)) / \sqrt{2} + \epsilon$$

$$e_{\text{right}} = (e_t + \text{proj}_d(e_t)) / \sqrt{2} + \epsilon$$

Where:

- d is a random direction vector derived from the seed
- $\text{proj}_d(e_t) = (d \cdot e_t) \cdot d$ is the projection of e_t onto d
- ϵ is small DP noise for privacy

Step 6: Re-Execution in TEEs

Both child shards re-execute the last 500 transactions on their projected state. This ensures consistency and detects any errors in the bisection.

Step 7: Erasure Coding

To ensure fault tolerance, shard data is encoded using Reed-Solomon erasure coding

with parameters $k = 5$ (data shards) and $m = 2$ (parity shards). This ensures that data remains available even if up to 2 out of 7 nodes in a shard go offline.

Step 8: Routing Update

The DHT and mixer routing tables are updated to route transactions to the child shards based on the sender/receiver hashes.

Step 9: Merge Protocol

If two sibling shards sustain < 10 TPS for 10 consecutive minutes, they automatically merge using the reverse bisection algorithm.

Mathematical Formalism

Sharding Perceptron Features

The sharding Perceptron input features include:

$X = [\text{TPS}, \text{mempool_size}, \text{queue_depth}, \text{gas_limit}, \text{cross_shard_ratio}, \text{time_of_day}]$

The output is:

$$p = \text{Sigmoid}(W_{\text{shard}} \cdot X + b_{\text{shard}}) = 1 / (1 + e^{-(W_{\text{shard}} \cdot X + b_{\text{shard}})})$$

Bisection Algorithm

The embedding $e_t \in \mathbb{R}^{64}$ is bisected as follows:

1. Generate random direction vector $d \in \mathbb{R}^{64}$:
 $d = \text{Normalize}(\text{BLAKE3}(\text{seed} || \text{"direction"}) \text{ as float})$
2. Compute projection:
 $\text{proj}_d(e_t) = (d \cdot e_t) \cdot d$
3. Compute child embeddings:
 $e_{\text{left}} = (e_t - \text{proj}_d(e_t)) / \sqrt{2} + \varepsilon$
 $e_{\text{right}} = (e_t + \text{proj}_d(e_t)) / \sqrt{2} + \varepsilon$

Where $\varepsilon \sim N(0, 0.1^2)$ is DP noise for privacy.

Re-Execution Verification

Both child shards re-execute the last N_{replay} transactions:

$$e_{\text{left}}^{(i+1)} = e_{\text{left}}^{(i)} + \sum_{\{tx \in \text{batch}_i\}} \delta_{tx}^{(\text{left})}$$

$$e_{\text{right}}^{(i+1)} = e_{\text{right}}^{(i)} + \sum_{\{tx \in \text{batch}_i\}} \delta_{tx}^{(\text{right})}$$

The outputs must match within tolerance:

$$\| e_{\text{left}}^{\text{final}} - e_{\text{left}}^{\text{expected}} \| \leq 10^{-6}$$

$$\| e_{\text{right}}^{\text{final}} - e_{\text{right}}^{\text{expected}} \| \leq 10^{-6}$$

Reed-Solomon Erasure Coding

For a block of data D of size S bytes:

1. Split D into $k = 5$ data shards of size S/k bytes each
2. Generate $m = 2$ parity shards using the Reed-Solomon encoding matrix:
$$P_i = \sum_{j=1}^k D_j \cdot \alpha_{i,j}$$

Where $\alpha_{i,j}$ are Vandermonde coefficients
3. The encoded data is $E = [D_1, D_2, \dots, D_k, P_1, P_2]$

To recover from 2 failures:

1. Select any $k = 5$ of the 7 shards
2. Invert the Vandermonde matrix to recover the original data:
$$D = V^{-1} \cdot E_{\text{selected}}$$

Merge Protocol

The reverse of bisection:

1. Compute the parent embedding:
$$e_{\text{parent}} = (e_{\text{left}} + e_{\text{right}}) / \sqrt{2}$$
2. Verify consistency:
$$\text{Verify}(e_{\text{parent}}, e_{\text{left}}, e_{\text{right}})$$
3. Update routing tables to merge the shards

Phase 9: Receipt & User Confirmation

Business/Functional Process

The final phase is the delivery of the Verifiable Delay Witness (VDW) to the user. The VDW is a permanent, offline-verifiable receipt that proves the transaction was included and executed correctly. Unlike traditional blockchain explorers, the VDW doesn't reveal any transaction details (amounts, counterparties, etc.) to third parties—it's a purely cryptographic proof that the transaction occurred.

Step 1: VDW Generation

After the block is finalized (Phase 7), the block producer (or a dedicated VDW issuer) generates a VDW for each transaction in the block.

The VDW contains:

1. Transaction Hash tx_hash (32 bytes): A SHA3-256 hash of the transaction's blinded payload
2. Halo2 Proof π (~400 bytes): Proves the homomorphic delta was computed correctly
3. Embedding Root $h_{\{t+1\}}$ (32 bytes): The new state root after the transaction
4. BLS Threshold Signature (48 bytes): Proves the block was finalized by a supermajority of validators
5. Dilithium-3 Signature (3,293 bytes): Proves the VDW was generated inside an attested validator

Step 2: VDW Distribution

The VDW is distributed to the user via:

1. HTTP/3 + QUIC: The user's wallet polls known nodes with `/vdw/:tx_hash` requests
2. GossipSub: The VDW is gossiped to all nodes in the network
3. Push Notifications: If the user's wallet is online, it may receive a push notification
4. Arweave/IPFS: The VDW is permanently pinned within 30 seconds of generation

Step 3: VDW Verification (Offline)

The user's wallet verifies the VDW offline in <30ms on modern hardware:

1. BLS Threshold Signature Check: Verify the aggregated signature against the DKG public key
2. Dilithium-3 Signature Check: Verify the validator's signature using the validator's public key
3. Halo2 Proof Verification: Run the Halo2 verifier on π
4. Homomorphic Root Reconciliation: Verify that $BLAKE3(e_{prev} + \delta_{tx}) = h_{\{t+1\}}$

Step 4: Balance Update

Upon successful verification, the wallet updates its local balance:

- Debit the spent notes (mark them as spent in the local database)
- Credit the output notes (add them to the local database of unspent notes)

Step 5: User Notification

The wallet displays a clear confirmation:

- "Transaction received: Z NERV from [blinded counterparty]"
- "Permanently confirmed forever"

The user can export the VDW as a QR code, .vdw file, or base64-encoded string for backup or third-party verification.

Step 6: Third-Party Verification (Selective Disclosure)

If the user needs to prove receipt to a third party (e.g., an exchange for an off-ramp, a tax authority, or a merchant), they can share the raw VDW or a derived ZK proof.

The third party:

1. Syncs the NERV light client (<100 KB forever)
2. Receives the VDW from the user
3. Runs the same offline verification as the wallet
4. Optionally verifies the amount (if the user provides a viewing key)

Mathematical Formalism

VDW Structure

The VDW is a structured binary blob:

$\text{VDW} = (\text{tx_hash}, \pi, h_{\{t+1\}}, \sigma_{\text{BLS}}, \sigma_{\text{Dilithium}}, \text{attestation})$

Where:

- $\text{tx_hash} = \text{BLAKE3}(\text{blinded_payload})$
- π is the Halo2 proof of correct delta computation
- $h_{\{t+1\}} = \text{BLAKE3}(e_{\{t+1\}})$
- $\sigma_{\text{BLS}} = \text{BLS.Agggregate}(\text{votes})$ (threshold signature)
- $\sigma_{\text{Dilithium}} = \text{Dilithium.Sign}(\text{sk_validator}, \text{VDW_without_sig})$
- attestation is the TEE attestation report (if applicable)

VDW Verification Algorithm (Pseudo-code)

text

```
fn verify_vdw(vdw: Vdw, trusted_state_root: H256) -> bool {
```

```
// 1. Verify BLS threshold signature

if !bls_verify(vdw.sigma_bls, vdw.h_t_plus_1, dkg_public_key) {

    return false;

}


// 2. Verify Dilithium signature

if !dilithium_verify(vdw.sigma_dilithium, vdw.payload,
validator_public_key) {

    return false;

}


// 3. Verify Halo2 proof

let delta = halo2_verify(vdw.pi);

if delta.is_none() {

    return false;

}


// 4. Verify homomorphic root reconciliation

let computed_root = blake3(trusted_state_root + delta);

if computed_root != vdw.h_t_plus_1 {

    return false;

}


return true;
```

}

Light Client Sync

A light client only needs to sync the embedding root chain:

headers = [h_0, h_1, h_2, ..., h_t]

Where each h_i is a 32-byte BLAKE3 hash. The total size is $t \times 32$ bytes, which is <100KB forever (for $t < 3,200$ blocks per day, $t \times 365 \approx 1.17\text{M}$ blocks, $1.17\text{M} \times 32 \approx 37\text{MB}$ after 1 year, but this is still manageable).

Offline Verification Using Light Client Data

To verify a VDW offline, the light client needs:

1. The trusted root h_t (cached from the last sync)
2. The delta δ_{tx} from the Halo2 proof
3. The VDW's claimed root h_{t+1}

The verification is:

$\text{BLAKE3}(h_t + \delta_{tx}) = h_{t+1}$

If this holds, the transaction is proven to be included in the state transition from block t to t+1.

Selective Disclosure (ZK Proof Derivation)

The user can generate a ZK proof from the VDW that proves a specific statement without revealing the full transaction:

For example, to prove "I received at least Z NERV":

1. The user knows the VDW and the decryption key for the output note
2. The user generates a ZK proof: $\pi_{\text{amount}} = \text{Prove}(\text{output_note.amount} \geq Z)$
3. The proof size is ~300-800 bytes and verifies in <50ms

VDW Archival

VDWs are permanently archived on Arweave and IPFS. After 5 years, old VDWs are aggregated into Merkle Mountain Range buckets for century-scale retrieval:

$\text{MMR} = \text{MerkleMountainRange}(\text{VDWs_year})$

A user can retrieve a VDW from 50 years ago using:

- 1. The transaction hash (remembered or stored)
- 2. The MMR proof (proof of inclusion in the MMR)
- 3. The Arweave transaction ID (for retrieval)

The total retrieval proof size is <1KB for century-scale verification.

Summary: The Full Transaction Lifecycle

Phase	Description	Time	Key Primitive
1	Client-Side Transaction Construction	<100ms	Halo2, ML-KEM-768
2	PQ-Sphinx Mixnet Routing	200-500ms	ML-KEM-768, ChaCha20
3	Mempool Integration & Gossip	<200ms	GossipSub, Merkle Tree
4	Threshold Decryption Ceremony	<500ms	BLS12-381, Feldman VSS
5	State Transition & NWO Execution	<50ms	NWO Perceptron

6	Self-Evolution (Adam & Huber)	<50ms	Adam, Huber Loss
7	AI-Native Consensus & Finality	<600ms	BLS, Monte Carlo
8	Dynamic Sharding & Erasure Coding	<4s	NWO, Reed-Solomon
9	Receipt & User Confirmation	<30ms	VDW, BLAKE3

Total End-to-End Latency: ~1.5 - 2 seconds for 10,000 transactions in a single shard (without shard splits). With sharding, the latency remains constant while throughput scales infinitely.

Note: Feldman VSS is a cryptographic building block, and DKG is the complete protocol built using it.

Here is the exact breakdown of how they differ and how they work together, especially in the context of NERV V2.0:

1. The Problem with a Single Dealer (Shamir's Secret Sharing)

To understand Feldman, we must start with Shamir's Secret Sharing (SSS). SSS allows a single "dealer" to split a secret s into n shares, such that any t shares can reconstruct s .

- The Flaw: SSS assumes the dealer is honest. A malicious dealer could distribute mathematically invalid shares. When the participants later tried to combine them, they wouldn't know who held the bad share, and the secret recovery would fail. Furthermore, in SSS, the dealer knows the secret s .

2. Feldman VSS (Verifiable Secret Sharing)

Feldman VSS was created to solve the "malicious dealer" flaw.

In Feldman VSS, the dealer still splits the secret s using a random polynomial, but they also broadcast public commitments to the coefficients of that polynomial.

- How it works: If the polynomial is $f(x) = a_0 + a_1x + a_2x^2$, the dealer broadcasts $C_0 = g^{(a_0)}, C_1 = g^{(a_1)}, C_2 = g^{(a_2)}$ to everyone.
- Verification: When participant i receives their share $f(i)$, they can use the public commitments and their own index i to mathematically verify: "Does $g^{(f(i))}$ equal $C_0 \cdot C_1^i \cdot C_2^{(i^2)}$?" If yes, the share is valid.
- The Remaining Flaw: While participants can now verify their shares, there is still a single trusted dealer who knows the master secret a_0 .

3. Distributed Key Generation (DKG)

In a decentralized blockchain like NERV, we cannot have a single trusted dealer generating the network's mempool encryption key. If we did, that dealer could secretly keep a copy of the private key and decrypt the entire mempool themselves!

DKG removes the dealer entirely.

A DKG protocol (like the one NERV V2.0 uses, which is based on Feldman VSS) works by having every validator act as a dealer simultaneously.

Here is the exact DKG flow used in NERV V2.0:

1. Everyone is a Dealer: Each validator v_i generates their own random secret s_i and creates a polynomial $f_i(x)$.
2. Feldman VSS Execution: Validator v_i uses Feldman VSS to distribute shares of s_i to all other validators, along with public commitments.
3. Verification: Every validator verifies the shares they received from every other validator using the Feldman commitments. If a share is invalid, they complain publicly.
4. Aggregation: Once all valid shares are distributed, each validator v_j sums up all the shares they received: $s_j = \sum f_i(j)$.
5. The Magic: The final collective public key PK is simply the sum of all the public commitments to the constant terms: $PK = \prod g^{(s_i)} = g^{(\sum s_i)}$.

Why this is critical for NERV V2.0

Because DKG uses Feldman VSS to let validators verify each other's shares without a central dealer:

- No single validator ever knows the full master secret key $s = \sum s_i$.
- The collective public key PK_DKG is public and used by wallets to encrypt the mempool.
- Only when a block producer assembles a batch do t validators come together, pool their individual secret shares s_i , and use Lagrange interpolation to perform a Threshold Decryption.

Summary:

Feldman VSS is the math that allows you to prove a secret share is valid. DKG is the protocol where everyone uses Feldman VSS at the same time so that no single person ever holds the master key.

Appendix C: Running a Node, Validator, and Relay

This appendix provides a comprehensive, step-by-step walkthrough for operating the three primary node types in the NERV V2.0 network: Full Nodes, Validator Nodes, and Sphinx Mixnet Relays. Each section covers the functional/business process, the underlying mathematics, and the exact commands required to participate in the network.

C.1 Node Types Overview

NERV V2.0 features a modular node architecture where participants can choose their role based on hardware capabilities, stake, and desired contribution to the network.

Node Type	Primary Function	Hardware Requirements	Earning Potential
-----------	------------------	-----------------------	-------------------

Full Node	Maintains a complete copy of the blockchain state, validates blocks, serves light clients	4+ vCPUs, 16GB+ RAM, 500GB+ SSD	None (voluntary)
Validator Node	Participates in AI-Native Consensus, performs threshold decryption, earns Useful-Work rewards via gradient submission	8+ vCPUs, 32GB+ RAM, 1TB+ SSD, GPU recommended	High (emissions + fees)
Sphinx Mixnet Relay	Routes private transactions through the 5-hop onion network, provides traffic-analysis resistance	2+ vCPUs, 8GB+ RAM, 100GB+ SSD	Medium (routing fees)

C.2 Full Node Operation

C.2.1 Functional/Business Process

A Full Node is the backbone of the NERV network. It maintains a complete, verified copy of the blockchain state—including all embedding roots, nullifier trees, commitment trees, and validator sets. Full Nodes do not participate in consensus or earn rewards, but they are essential for network decentralization and provide critical infrastructure for light clients and other nodes.

Step 1: Installation & Initialization

The operator downloads the NERV binary from the official repository, installs dependencies, and generates a default configuration file. The configuration file specifies the data directory, network listening addresses, and privacy parameters.

Step 2: Network Synchronization

Upon first startup, the Full Node initiates a peer-to-peer (P2P) synchronization process. It connects to bootstrap nodes, discovers peers via the Kademlia DHT, and downloads the complete blockchain history.

The synchronization process validates each piece of data as it arrives, ensuring the node only accepts blocks that satisfy all consensus rules.

Step 3: Continuous Operation

Once synchronized, the Full Node:

- Gossips new transactions and blocks to peers
- Serves historical data to light clients via HTTP/3 + QUIC
- Maintains the DHT for peer discovery
- Stores VDWs for permanent archival

Step 4: Light Client Support

Full Nodes expose a `/vdw/:tx_hash` endpoint that allows light clients to retrieve Verifiable Delay Witnesses for their transactions. This enables mobile and resource-constrained wallets to verify transactions without downloading the full chain.

C.2.2 Mathematical Formalism

State Representation

A Full Node maintains the complete state at height t^* :

$$S_t = (e_t, \text{nullifier_root}_t, \text{commitment_root}_t, \text{validator_set}_t)$$

Where:

- $e_t \in \mathbb{R}^{64}$ is the embedding vector (512 bytes)
- `nullifier_root_t` is the Merkle root of the nullifier tree
- `commitment_root_t` is the Merkle root of the commitment tree
- `validator_set_t` is the set of active validators

Block Validation

For each received block, the Full Node verifies:

1. State Root Consistency:

$$\text{BLAKE3}(e_{\{t+1\}} || \text{nullifier_root}_{\{t+1\}} || \text{commitment_root}_{\{t+1\}} || \text{validator_set}_{\{t+1\}}) = h_{\{t+1\}}$$
2. BLS Threshold Signature Verification:

$$e(\text{agg_sig}, g) = e(\text{pk_agg}, h_{\{t+1\}})$$
3. Weight Update Verification:

$$h_W = \text{BLAKE3}(W_{\text{new}})$$

$$\sum_{\{v: h_{W,v} = h_W, \text{majority}\}} \text{stake}_v \geq 0.67 \times \sum_{\{v \in V\}} \text{stake}_v$$

VDW Serving

When a light client requests a VDW via `/vdw/:tx_hash`, the Full Node retrieves the witness from its local storage and serves it as a binary blob:

`VDW = (tx_hash,  $\pi$ ,  $h_{\{t+1\}}$ ,  $\sigma_{\text{BLS}}$ ,  $\sigma_{\text{Dilithium}}$ , attestation)`

C.2.3 Commands

Step 1: Clone and Build

```
bash
git clone https://github.com/nerv-bit/nerv.git
cd nerv
cargo build --release
```

Step 2: Generate Configuration

```
bash
./target/release/nerv --generate-config
```

This creates a `nerv.toml` file in the current directory. Key configuration parameters include:

```
toml
```

```
[network]
```

```
listen_addresses = ["/ip4/0.0.0.0/tcp/9000"]
```

```
bootstrap_nodes = ["/dns/bootstrap.nerv.net/tcp/9000/p2p/..."]
```

```
[storage]
```

```
data_dir = "./nerv_data"
```

```
max_block_history = 1000000
```

```
[privacy]
```

```
enable_mixnet = false # Full nodes don't participate in the mixnet
```

Step 3: Start the Full Node

```
bash
```

```
./target/release/nerv --config nerv.toml
```

Step 4: Monitor Synchronization

```
bash
```

```
./target/release/nerv --status
```

Expected output:

```
text
```

```
Block Height: 1,234,567
```

```
Peers: 47
```

```
State Root: 0x7f8e9d...
```

```
Sync Progress: 100.00%
```

Step 5: Serve VDWs (Automatic)

Once synchronized, the node automatically serves VDWs via HTTP/3:

```
bash
```

```
curl https://localhost:9000/vdw/0xabc123...
```

C.3 Validator Node Operation

C.3.1 Functional/Business Process

A Validator Node is a Full Node that additionally participates in NERV's AI-Native Consensus and Useful-Work economy. Validators stake NERV tokens, propose blocks, vote on state transitions, compute gradients for the NWO Perceptron, and earn rewards proportional to their contributions.

Step 1: Staking & Registration

The operator must stake a minimum amount of NERV tokens (e.g., 100,000 NERV) to become a validator. The stake is locked in a smart contract and can be slashed for misbehavior.

The operator generates a validator keypair (Dilithium-3 for signing, BLS12-381 for consensus voting) and registers their public keys on-chain.

Step 2: Block Proposal (when selected)

Validators are selected as block producers via a stake-weighted VRF mechanism. When selected, the validator:

1. Selects a batch of encrypted transactions from the mempool
2. Coordinates the threshold decryption ceremony
3. Executes the state transition (Phase 5 of the transaction lifecycle)
4. Computes the Adam gradient update (Phase 6)

5. Proposes the block to the network

Step 3: Consensus Voting

For every block, validators:

1. Compute the Perceptron prediction: $e_pred = W_new \cdot (e_t + \Delta_B) + b$
2. Generate a BLS signature on $h_pred = \text{BLAKE3}(e_pred)$
3. Broadcast the vote to the network

If $\geq 67\%$ of weighted stake agrees on the same hash, the block achieves probabilistic finality in $\sim 600\text{ms}$.

Step 4: Gradient Computation & Useful-Work

Every block, validators compute the Huber loss and Adam gradient:

$$L_\delta(y, \hat{y}) = \sum_{j=1}^{64} L_\delta(y_j, \hat{y}_j)$$

$$g = \nabla_W L_\delta(y, \hat{y})$$

They then update the weights using the Adam optimizer and broadcast the new weight hash. Validators who submit gradients that reduce the loss are eligible for Useful-Work rewards from the emission pool.

Step 5: Reward Collection

Validators earn rewards from:

1. Block Proposal Fees: Transaction fees from included transactions
2. Useful-Work Emissions: Newly minted NERV tokens proportional to gradient contribution
3. Voting Rewards: Share of emissions for honest voting

C.3.2 Mathematical Formalism

Validator Registration

The validator's stake is recorded on-chain:

$$\text{stake}_v \geq \text{min_stake}$$

The validator's public keys are registered:

$$\text{pk_Dilithium}_v, \text{pk_BLS}_v$$

DKG Participation

Each validator participates in the Distributed Key Generation protocol:

$$s_j = \sum_{i=1}^n f_i(j)$$

Where $f_i(x)$ is the polynomial generated by validator i . The collective public key is:

$$PK_{DKG} = g^{(\sum s_i)} = g^s$$

Block Proposal Selection

The block producer is selected via VRF:

$$\text{selector} = \text{VRF}(\text{sk}_{\text{BLS}_v}, \text{block_height}, \text{nonce})$$

$$\text{producer} = \text{argmax}_{v \in V} (\text{stake}_v \times \text{reputation}_v \times \text{selector})$$

Weighted Quorum

The voting weight for validator v is:

$$w_v = \text{stake}_v \times \text{reputation}_v$$

Quorum is reached if:

$$(\sum_{v \in V_{\text{agree}}} w_v) / (\sum_{v \in V} w_v) \geq 0.67$$

Gradient Computation

The gradient of the Huber loss with respect to the weights is:

$$g_{\{j,k\}} = (\partial L_{\delta} / \partial \hat{y}_j) \cdot x_k$$

Where:

$$\partial L_{\delta} / \partial \hat{y}_j = \{ \hat{y}_j - y_j \text{ if } |y_j - \hat{y}_j| \leq \delta ; -\delta \cdot \text{sign}(y_j - \hat{y}_j) \text{ otherwise} \}$$

Adam Weight Update

The validator computes:

$$m_{\{j,k\}} \leftarrow \beta_1 \cdot m_{\{j,k\}} + (1 - \beta_1) \cdot g_{\{j,k\}}$$

$$v_{\{j,k\}} \leftarrow \beta_2 \cdot v_{\{j,k\}} + (1-\beta_2) \cdot g_{\{j,k\}}^2$$

$$m_{\text{hat}}_{\{j,k\}} = m_{\{j,k\}} / (1 - \beta_1^t)$$

$$v_{\text{hat}}_{\{j,k\}} = v_{\{j,k\}} / (1 - \beta_2^t)$$

$$W_{\text{new}}_{\{j,k\}} = W_{\{j,k\}} - \alpha \cdot (m_{\text{hat}}_{\{j,k\}} / (\sqrt{v_{\text{hat}}_{\{j,k\}}} + \epsilon))$$

Useful-Work Reward

$$\text{reward}_v = \text{pool} \times (\text{loss_reduction}_v / \sum_i \text{loss_reduction}_i)$$

Where:

$$\text{loss_reduction}_v = L_{\delta}(y, \hat{y}_{\text{old}}) - L_{\delta}(y, \hat{y}_{\text{new}_v})$$

Slashing Conditions

A validator is slashed (stake reduced by 0.5% to 5%) for:

1. Double Signing: Signing two different blocks at the same height
2. Invalid Gradient Submission: Submitting a gradient that increases the loss
3. Invalid Partial Decryption: Submitting an incorrect partial decryption during threshold decryption
4. Liveness Failure: Missing > 100 consecutive block proposals or votes

The slashed amount is:

$$\text{slash_amount} = \text{stake}_v \times \text{slash_rate}$$

C.3.3 Commands

Step 1: Generate Validator Keypair

```
bash
```

```
./target/release/nerv --generate-validator-keys
```

This creates:

- `validator_sk_dilithium.pem` (private signing key)
- `validator_pk_dilithium.pem` (public verification key)

- `validator_sk_bls.pem` (private BLS key)
- `validator_pk_bls.pem` (public BLS key)

Step 2: Configure Validator Settings

Edit `nerv.toml` to add validator configuration:

```
toml

[validator]

enabled = true

stake_amount = 100000

commission_rate = 0.10 # 10% commission on delegator rewards

seed_phrase = "your_24_word_mnemonic" # Or use --seed-phrase flag


[dkg]

threshold = 67 # Percentage of validators required for decryption
```

Step 3: Stake Tokens

```
bash

./target/release/nerv --stake --amount 100000 --validator-pk
./validator_pk_bls.pem
```

This sends a transaction to the staking contract, locking the tokens and registering the validator.

Step 4: Start the Validator Node

```
bash

./target/release/nerv --validator --config nerv.toml
```

Or using the seed phrase directly:

```
bash
```

```
./target/release/nerv --validator --seed-phrase "your_24_word_mnemonic"
```

Step 5: Monitor Validator Status

```
bash
```

```
./target/release/nerv --validator-status
```

Expected output:

```
text
```

```
Validator ID: 0x7f8e9d...
```

```
Stake: 100,000 NERV
```

```
Reputation: 0.95
```

```
Last Proposed Block: 1,234,567
```

```
Gradient Contribution: 0.023%
```

```
Total Rewards: 1,234.56 NERV
```

```
Slashing Risk: Low
```

Step 6: View Gradient Performance

```
bash
```

```
./target/release/nerv --gradient-stats
```

Expected output:

```
text
```

```
Current Loss: 0.000123
```

```
Loss Reduction: 0.000045 (36.6%)
```

```
Adam Step: 1,234,567
```

```
Weight Hash: 0xabc123...
```

C.4 Sphinx Mixnet Relay Operation

C.4.1 Functional/Business Process

A Sphinx Mixnet Relay is a specialized node that participates in NERV's 5-hop onion-routing privacy layer. Relays receive encrypted packets, peel one layer of encryption, add timing jitter and cover traffic, and forward the inner packet to the next relay. Relays earn routing fees for their service.

Step 1: Relay Registration

The operator registers their relay with the on-chain relay registry by posting a bond (stake) that can be slashed for misbehavior. The registry is maintained via governance votes and provides the list of relays used by wallets for onion construction.

Step 2: Packet Reception

A relay receives an encrypted Sphinx packet from the previous hop (or from a wallet for the first hop). The packet contains:

- An ML-KEM-768 ciphertext (`ct_i`)
- An encrypted layer (`Layer_i`)
- A Sphinx header

Step 3: Decapsulation & Decryption

The relay uses its ML-KEM-768 private key to decapsulate the shared secret:

```
ss_i = MLKEM768.Decapsulate(sk_i, ct_i)
```

It then derives the symmetric key:

```
key_i = BLAKE3.KDF(ss_i, domain_separator || i)
```

And decrypts the layer:

```
plaintext_i = ChaCha20Poly1305.Decrypt(key_i, nonce_i, Layer_i, AD_i)
```

Step 4: Routing Extraction

The relay extracts the next hop's address and the inner payload:

$(R_{\{i+1\}}, Layer_{\{i+1\}}) = plaintext_i$

Step 5: Timing Jitter & Cover Traffic

To defeat traffic analysis, the relay:

1. Waits for a delay sampled from an exponential distribution: $\Delta t \sim \text{Exp}(\mu = 100\text{ms}, \sigma = 200\text{ms})$
2. Generates dummy packets using a local NWO Perceptron that mimics real traffic distributions
3. Mixes real and dummy packets to prevent volume correlation

Step 6: Forwarding

The relay forwards $Layer_{\{i+1\}}$ to $R_{\{i+1\}}$ via libp2p, using the Noise Protocol Framework with PQ-NOISE extensions (ML-KEM-768 + ChaCha20-Poly1305).

Step 7: Reward Collection

Relays earn routing fees from:

1. Per-Hop Fees: A small fee paid by the wallet for each hop
2. Cover Traffic Fees: Additional fees for generating dummy traffic
3. QoS Bonuses: Bonuses for high-quality service (low latency, high uptime)

C.4.2 Mathematical Formalism

ML-KEM-768 Key Encapsulation

For each packet, the relay performs:

$ss_i = \text{MLKEM768.Decapsulate}(sk_i, ct_i)$

Where:

- sk_i is the relay's private key (never leaves the relay)
- ct_i is the ciphertext from the packet
- ss_i is the recovered shared secret

Symmetric Key Derivation

$key_i = \text{BLAKE3.KDF}(ss_i, domain_separator || i || nonce)$

ChaCha20-Poly1305 Decryption

```
plaintext_i = ChaCha20Poly1305.Decrypt(key_i, nonce_i, Layer_i, AD_i)
```

Timing Jitter

The delay is sampled from:

```
 $\Delta t \sim \text{Exp}(\mu = 100\text{ms}, \sigma = 200\text{ms})$ 
```

The probability density function is:

```
 $f(\Delta t) = (1/\sigma) \cdot e^{(-\Delta t / \sigma)}$ 
```

Cover Traffic Generation

The relay's local NWO Perceptron generates dummy packets:

```
 $\hat{y}_{\text{dummy}} = W_{\text{dummy}} \cdot X_{\text{traffic}} + b_{\text{dummy}}$ 
```

Where:

- W_{dummy} is the relay's local weight matrix
- X_{traffic} are features of recent traffic (size, timing, destination entropy)
- b_{dummy} is the bias term

The Perceptron is updated in real-time using the Adam optimizer to minimize:

```
 $L_{\text{dummy}} = || \hat{y}_{\text{dummy}} - y_{\text{actual}} ||^2$ 
```

Reward Calculation

The routing fee for relay i is:

```
 $\text{fee}_i = \text{base\_fee} \times (1 + \text{QoS\_multiplier})$ 
```

Where QoS_multiplier depends on:

- Uptime ($\text{uptime} \in [0, 1]$)
- Latency ($\text{latency} \in [0, 1]$)
- Packet loss rate ($\text{loss} \in [0, 1]$)

```
 $\text{QoS\_multiplier} = 0.5 \times \text{uptime} + 0.3 \times (1 - \text{latency}) + 0.2 \times (1 - \text{loss})$ 
```

Slashing Conditions

A relay is slashed for:

1. Packet Dropping: Deliberately dropping packets (detected via cryptographic proofs)
2. Timing Correlation: Failing to add proper jitter (detected via statistical analysis)
3. Invalid Decryption: Submitting incorrect decryption proofs

C.4.3 Commands

Step 1: Generate Relay Keypair

```
bash
```

```
./target/release/nerv --generate-relay-keys
```

This creates:

- `relay_sk_mlkem.pem` (private ML-KEM key)
- `relay_pk_mlkem.pem` (public ML-KEM key)
- `relay_sk_dilithium.pem` (private Dilithium signing key)
- `relay_pk_dilithium.pem` (public Dilithium verification key)

Step 2: Configure Relay Settings

Edit `nerv.toml` to add relay configuration:

```
toml
```

```
[relay]
```

```
enabled = true
```

```
bond_amount = 10000
```

```
jitter_mu = 100 # Base delay in milliseconds
```

```
jitter_sigma = 200 # Jitter strength in milliseconds
```

```
cover_traffic_ratio = 0.5 # Ratio of dummy to real packets
```

```
[mixnet]
```

```
max_hops = 5
```

```
packet_size = 2048 # Fixed packet size in bytes
```

Step 3: Register the Relay

```
bash
```

```
./target/release/nerv --register-relay --bond 10000 --relay-pk  
./relay_pk_mlkem.pem
```

This sends a transaction to the relay registry, posting the bond and registering the relay's public key.

Step 4: Start the Relay Node

```
bash
```

```
./target/release/nerv --relay --config nerv.toml
```

Or using the seed phrase directly:

```
bash
```

```
./target/release/nerv --relay --seed-phrase "your_24_word_mnemonic"
```

Step 5: Monitor Relay Status

```
bash
```

```
./target/release/nerv --relay-status
```

Expected output:

```
text
```

```
Relay ID: 0x7f8e9d...
```

```
Bond: 10,000 NERV
```

```
Uptime: 99.97%
```

```
Packets Routed: 1,234,567
```

```
Average Latency: 42 ms
```


Cover Traffic Generated: 617,283 packets

Total Fees Earned: 12,345.67 NERV

Slashing Risk: Low

Step 6: View Traffic Statistics

bash

```
./target/release/nerv --relay-stats
```

Expected output:

text

Real Packets: 1,234,567

Dummy Packets: 617,283

Packet Drop Rate: 0.02%

Average Jitter: 198 ms

QoS Score: 0.97

Step 7: Withdraw Fees

bash

```
./target/release/nerv --relay-withdraw --amount 1000
```

C.5 Summary: Node Types Comparison

Aspect	Full Node	Validator Node	Sphinx Mixnet Relay
--------	-----------	----------------	---------------------

Primary Role	State replication & serving	Consensus & Useful-Work	Privacy routing
Stake Required	None	100,000+ NERV	10,000+ NERV
Earning Potential	None	High (emissions + fees)	Medium (routing fees)
Hardware Requirements	Moderate	High (GPU recommended)	Low
Slashing Risk	None	Yes (0.5%-5%)	Yes (bond loss)
Key Primitives	BLAKE3, Merkle Trees	Halo2, Adam, BLS	ML-KEM-768, ChaCha20
Command	<code>nerv --config</code>	<code>nerv --validator</code>	<code>nerv --relay</code>

The future is learned, private, and fast.

References

NIST Post-Quantum Cryptography Standards

- [1] National Institute of Standards and Technology. *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)*. August 2024.
 - [2] National Institute of Standards and Technology. *FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA / CRYSTALS-Dilithium)*. August 2024.
 - [3] National Institute of Standards and Technology. *FIPS 205: Stateless Hash-Based Digital Signature Standard (SLH-DSA / SPHINCS+)*. August 2024.
-

Zero-Knowledge Proof Systems

- [4] Electric Coin Company. *Halo 2: Recursive Proof Composition without Trusted Setup*. 2020–2025. <https://github.com/zcash/halo2>[reference:3]
 - [5] S. Bowe, J. Grigg, D. Hopwood. *Nova: Recursive SNARKs without Trusted Setup*. Microsoft Research, 2022–2025. <https://github.com/microsoft/Nova>[reference:4]
-

Machine Learning & Optimization

- [6] D. P. Kingma and J. Ba. *Adam: A Method for Stochastic Optimization*. 3rd International Conference on Learning Representations (ICLR), 2015.
- [7] P. J. Huber. *Robust Estimation of a Location Parameter*. Annals of Mathematical Statistics, 1964.
- [8] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, L. Zhang. *Deep Learning with Differential Privacy*. ACM CCS, 2016.
- [9] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, B. A. y Arcas. *Communication-Efficient Learning of Deep Networks from Decentralized Data*. AISTATS, 2017.

Verifiable Secret Sharing & Distributed Key Generation

[10] P. Feldman. *A Practical Scheme for Non-interactive Verifiable Secret Sharing*. 28th Annual Symposium on Foundations of Computer Science (FOCS), IEEE, 1987, pp. 427–437.

[11] T. P. Pedersen. *Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing*. Advances in Cryptology – CRYPTO '91, 1991, pp. 129–140.

[12] T. P. Pedersen. *Distributed Key Generation Protocol (Joint-Feldman DKG)*. 1991.

Anonymous Routing & Mixnets

[13] G. Danezis and I. Goldberg. *Sphinx: A Compact and Provably Secure Mix Format*. IEEE Symposium on Security and Privacy, 2009.

[14] A. Rial, A. M. Piotrowska. *Outfox: A Postquantum Packet Format for Layered Mixnets*. WEPS, 2025.

[15] C. Diaz, A. Rial, A. M. Piotrowska, A. Kiayias. *Nym Mixnet Whitepaper*. Nym Technologies, 2021.

Erasure Coding & Data Availability

[16] I. S. Reed and G. Solomon. *Polynomial Codes over Certain Finite Fields*. Journal of the Society for Industrial and Applied Mathematics, Vol. 8, No. 2, 1960, pp. 300–304.

Networking & Peer-to-Peer Protocols

[17] libp2p Project. *libp2p: A Modular Network Stack*. <https://libp2p.io/reference:15>

[18] D. Maymounkov and P. Mazières. *Kademlia: A Peer-to-Peer Information System Based on the XOR Metric*. IPTPS, 2002.

[19] libp2p GossipSub Specification. *GossipSub: A P2P Pubsub Routing Protocol*. <https://github.com/libp2p/specs/tree/master/pubsub/gossipsub>[reference:16]

Secure Communication Protocols

[20] T. Perrin. *The Noise Protocol Framework*. 2018. <https://noiseprotocol.org/noise.pdf>[reference:17]

[21] PQNoise. *Post-Quantum Cryptography in the Noise Protocol Framework*. 2025.

[22] Y. Nir and A. Langley. *RFC 8439: ChaCha20 and Poly1305 for IETF Protocols*. IETF, June 2018.

Cryptographic Hash Functions

[23] J. O'Connor, et al. *BLAKE3: A Cryptographic Hash Function*. 2020.

[24] NIST. *FIPS 202: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*. 2015.

Elliptic Curve Cryptography

[25] S. Bowe. *BLS12-381: A Pairing-Friendly Elliptic Curve Construction*. 2017.

[26] P. S. L. M. Barreto, B. Lynn, M. Scott. *Constructing Elliptic Curves with Prescribed Embedding Degrees*. Cryptology ePrint Archive, Report 2002/088.

Game Theory & Incentive Mechanisms

[27] L. S. Shapley. *A Value for n -Person Games*. In Contributions to the Theory of Games, Vol. II, Princeton University Press, 1953.

Federated Learning & Secure Aggregation

[28] K. Bonawitz, et al. *Practical Secure Aggregation for Privacy-Preserving Machine Learning*. ACM CCS, 2017.

[29] P. Kairouz, et al. *Advances and Open Problems in Federated Learning*. Foundations and Trends in Machine Learning, 2021.

Consensus & Blockchain Protocols

[30] V. Buterin. *Ethereum: A Secure Decentralised Generalised Transaction Ledger (Yellow Paper)*. 2014.

[31] Solana Labs. *Solana Consensus (Tower BFT)*. 2024.

[32] Aptos Labs. *AptosBFT: Practical Byzantine Fault Tolerance*. 2023.

[33] Sui Foundation. *Mysticeti: Low-Latency DAG Consensus*. 2024.

Privacy-Preserving Blockchain Protocols

[34] Monero Research Lab. *RingCT 3.0: Succinct Confidential Transactions*. 2020.

[35] Zcash Foundation. *Zcash Protocol Specification (Sapling & Orchard)*. 2020.

[36] Penumbra Labs. *Penumbra: Shielded Transactions on Cosmos*. 2023–2025.

Verifiable Delay Functions & Light Clients

[37] B. Bünz, et al. *FlyClient: Super-Light Client for PoW Blockchains*. Cryptology ePrint Archive, Report 2019/226.

[38] B. Wesolowski. *Verifiable Delay Functions*. Cryptology ePrint Archive, Report 2018/601.

[39] K. Pietrzak. *Verifiable Delay Functions*. Cryptology ePrint Archive, Report 2018/627.

Zero-Knowledge Proofs & SNARKs

[40] A. Gabizon, Z. J. Williamson, O. Ciobotaru. *PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge*. Cryptology ePrint Archive, Report 2019/953.

[41] H. Bünz, et al. *Bulletproofs: Short Proofs for Confidential Transactions*. IEEE S&P, 2018.

[42] S. Bowe, A. Chiesa, M. Green, I. Miers, P. Mishra, H. Wu. *Marlin: Preprocessing zkSNARKs with Universal and Updatable SRS*. Cryptology ePrint Archive, Report 2019/1047.

Dynamic Sharding & Scalability

[43] V. Buterin. *Danksharding Specification (EIP-4844)*. 2023.

[44] Polygon Technology. *Polygon zkEVM Technical Documentation*. 2024.

[45] zkSync Era. *Hyperchains & Boojum Proof System*. 2024.

[46] Scroll. *zkEVM Whitepaper*. 2024.

Permanent Storage & Archival

[47] Arweave. *Arweave: A Permanently Decentralized Data Store*. 2024.

[48] Protocol Labs. *IPFS: InterPlanetary File System*. 2025.

Formal Verification & Security Analysis

[49] B. Blanchet. *ProVerif: Cryptographic Protocol Verifier*.
<https://prosecco.gforge.inria.fr/personal/bblanche/proverif>

[50] L. Lamport. *TLA+ Specification Language*.
<https://lamport.azurewebsites.net/tla/tla.html>

[51] Lean Theorem Prover. <https://lean-lang.org>

Transaction Lifecycle & Cryptographic Protocols

[52] S. Noether. *Ring Confidential Transactions*. Monero Research Lab, 2015.

[53] E. Ben-Sasson, et al. *ZeroCash: Decentralized Anonymous Payments from Bitcoin*. IEEE S&P, 2014.

[54] M. Pilkington. *Blockchain Technology: Principles and Applications*. Research Handbook on Digital Transformations, 2016.

Neural Networks & Deep Learning

[55] A. Vaswani, et al. *Attention Is All You Need*. NeurIPS, 2017.

[56] N. Shazeer. *GLU Variants Improve Transformer*. arXiv:2002.05202, 2020.

[57] J. Ainslie, et al. *Grouped-Query Attention*. arXiv:2305.13245, 2023.

Differential Privacy

[58] C. Dwork, A. Roth. *The Algorithmic Foundations of Differential Privacy*. Foundations and Trends in Theoretical Computer Science, 2014.

[59] M. Abadi, et al. *Deep Learning with Differential Privacy*. ACM CCS, 2016.

Cryptographic Libraries & Implementations

[60] PQCclean Project. *PQCclean: Clean, Portable, Tested Implementations of Post-Quantum Cryptography*. <https://github.com/PQCclean/PQCclean>

[61] liboqs Project. *Open Quantum Safe: Post-Quantum Cryptography for TLS*. <https://openquantumsafe.org>

[62] RustCrypto. *Rust Implementation of Cryptographic Algorithms*. <https://github.com/RustCrypto>

Distributed Systems & Consensus

[63] L. Lamport, R. Shostak, M. Pease. *The Byzantine Generals Problem*. ACM Transactions on Programming Languages and Systems, 1982.

[64] M. Castro, B. Liskov. *Practical Byzantine Fault Tolerance*. USENIX OSDI, 1999.

[65] E. Buchman. *Tendermint: Byzantine Fault Tolerance in the Age of Blockchains*. PhD Thesis, 2016.